DC MOTOR CONTROL STUDY NOTES

Feedback (Incremental Encoders) & Actuation (PWM)

Topic Overview

This module covers the core principles of closed-loop DC motor control. It addresses two fundamental engineering challenges:

1. **State Estimation (The "Eyes"):** Using incremental optical quadrature encoders to determine precise angular position, velocity, and direction of rotation.
2. **Actuation (The "Muscle"):** Using Pulse Width Modulation (PWM) and H-Bridge drivers to control motor speed and direction using purely digital microcontrollers.

Core Concepts

- **Incremental Encoder:** An electromechanical device that outputs digital pulses corresponding to relative mechanical displacement. It does not know absolute global position natively.
- **Quadrature Phase Shift:** The generation of two square waves (Channels A and B) offset by $90°$ electrical degrees to decode directional movement.
- **Decoding Resolution (X1, X2, X4):** Techniques defining how many waveform transitions are tracked per physical slot to multiply positional accuracy.
- **Pulse Width Modulation (PWM):** A modulation technique that varies the width of digital pulses to simulate a variable analog voltage across an inductive load.
- **H-Bridge Driver:** A transistor-based circuit that permits bidirectional current flow to control a motor's rotational direction while isolating low-power control circuitry from high-power loads.

Detailed Notes

Part I: Angular Position & Direction Measurement

1. Hardware Mechanism

An optical incremental encoder features a slotted disc attached to the motor shaft. As the shaft rotates, the disc alternately blocks and passes light emitted from an LED toward a photodetector. This mechanism converts continuous mechanical rotation into a discrete digital **square wave**.

2. The Quadrature Technique

A single sensor channel can measure speed and distance but cannot determine direction, as its square wave looks identical in both clockwise (CW) and counter-clockwise (CCW) rotations.

By adding a second sensor channel physically offset by $90°$ electrical degrees, we generate two signals: **Channel A** and **Channel B**. The sequence of transitions forms a 2-bit **Gray Code** (where only one bit changes state at a time).

```
Clockwise (CW) Cycle:         00 → 10 → 11 → 01 → 00
Counter-Clockwise (CCW) Cycle: 00 → 01 → 11 → 10 → 00
```

3. Quadrature Decoding Levels

- **X1 Decoding:** Triggers only on a single edge (e.g., rising edge) of Channel A.
  $$Total\ Counts = Physical\ Slots\ (PPR)$$
- **X2 Decoding:** Triggers on both the rising and falling edges of Channel A.
  $$Total\ Counts = PPR \times 2$$
- **X4 Decoding:** Triggers on all rising and falling edges of *both* Channel A and Channel B. This yields maximum precision.
  $$Total\ Counts = PPR \times 4$$

4. Microcontroller Interfacing & Counting Logic

To process these rapid transitions without dropping steps, microcontrollers use two main software architectures:

Method A: External Software Interrupts (GPIO)

Pins are configured to detect voltage transitions (`CHANGE`) and immediately execute an Interrupt Service Routine (ISR).

> **X2 Decoding Rule (Tracking Pin A Transitions Only):**
>
> - If Current State of A ≠ Current State of B → **Clockwise (+1)**
> - If Current State of A = Current State of B → **Counter-Clockwise (-1)**

> **X4 Decoding Rule (Tracking Both Pin A and Pin B Transitions):**
>
> - When **Channel A** changes: If $A \neq B$ → CW (+1), else → CCW (-1)
> - When **Channel B** changes: If $A == B$ → CW (+1), else if $A \neq B$ → CCW (-1)

Method B: Hardware QEI (Quadrature Encoder Interface)

Advanced microcontrollers (e.g., STM32, ESP32) utilize dedicated internal hardware timers routed to encoder pins. The hardware increments or decrements an internal register on every X4 transition completely in the background, consuming 0% CPU overhead.

Part II: Pulse Width Modulation (PWM)

1. The Core Concept

Digital microcontrollers can only output distinct high or low voltage levels (e.g., 0V or 5V). To alter motor speed, the microcontroller must deliver intermediate power states. PWM achieves this by switching the digital pin ON and OFF at a high frequency. Because the inductive coils inside a DC motor possess mechanical and electrical inertia, they integrate these high-speed voltage pulses over time, reacting to the **average voltage** rather than individual pulses.

2. Duty Cycle Mechanics

The **Duty Cycle** represents the percentage of time a signal is HIGH (T_{on}) relative to the total period (T) of one cycle.

$$Duty\ Cycle = (T_{on} / (T_{on} + T_{off})) \times 100\%$$

The effective average voltage (V_{avg}) delivered to the motor is directly proportional to this percentage:

$$V_{avg} = Duty\ Cycle \times V_{max}$$

3. Power Amplification & H-Bridges

Microcontroller GPIO pins typically source less than 40mA of current. DC motors require hundreds of milliamperes to several amperes. An **H-Bridge** circuit acts as an electronic valve network. It uses low-power PWM control signals to switch high-current external power sources across the motor terminal. By activating diagonally opposite pairs of switches (MOSFETs or transistors) within the "H" configuration, the driver reverses the polarity of the applied voltage, controlling both motor speed and direction.

Definitions

- **PPR (Pulses Per Revolution):** The number of physical slots or full square-wave cycles per channel in one complete mechanical revolution of the encoder disk.
- **CPR (Counts Per Revolution):** The net resolution available after quadrature decoding. For X4 decoding, $CPR = PPR \times 4$.

- **Volatile (Keyword):** A compiler directive indicating that a variable can change unexpectedly outside the sequential execution flow (e.g., inside an ISR), preventing the compiler from optimizing it out or caching it.
- **Atomic Operation:** An uninterrupted processor action. If a multi-byte variable is read on an 8-bit architecture, the process must be temporarily made atomic by disabling interrupts to prevent data corruption.
- **Frequency (f):** The number of completed PWM cycles per second, measured in Hertz (Hz). Higher frequencies diminish audible motor humming.

Important Formulas / Rules

1. Quadrature State Logic Matrix (X4 Decoding)

$$\text{Direction} = \text{CW } (+1) \text{ if } (\Delta A \text{ and } A \neq B) \text{ or } (\Delta B \text{ and } A == B)$$
$$\text{Direction} = \text{CCW } (-1) \text{ if } (\Delta A \text{ and } A == B) \text{ or } (\Delta B \text{ and } A \neq B)$$

2. Angular Position Calculation (θ)

To extract the exact mechanical angle in degrees given the total encoder pulse count (N):

$$\theta = (N / (\text{PPR} \times \text{Decoding Multiplier} \times \text{Gear Ratio})) \times 360°$$

3. Average PWM Voltage Equation

$$V_{avg} = (\text{Register Value} / 255) \times V_{max} \quad \text{(For 8-bit Arduino Registers)}$$

Examples

1. Angular Position Calculation Walkthrough

Problem: A motor utilizes a 500 PPR encoder paired with a 1:30 reduction gearbox. The system implements X4 software decoding. If the microcontroller register records a net count of 15,000 pulses, what is the output shaft's position?

Solution:

1. **Determine total CPR:** $CPR = 500 \times 4 = 2000 \text{ counts/rev}$
2. **Account for gearbox:** $\text{Total CPR} = 2000 \times 30 = 60{,}000 \text{ counts/rev}$
3. **Calculate angle:**
   $\theta = (15{,}000 / 60{,}000) \times 360° = 0.25 \times 360° = 90°$

2. Complete Arduino X4 Encoder Tracking Template

```
// Volatile prevents compiler registry caching optimization
volatile long encoderCount = 0;

const int pinA = 2; // External Interrupt 0
const int pinB = 3; // External Interrupt 1

void setup() {
  Serial.begin(9600);

  pinMode(pinA, INPUT_PULLUP);
  pinMode(pinB, INPUT_PULLUP);

  // Attach ISR to catch EVERY rising and falling transition
  attachInterrupt(digitalPinToInterrupt(pinA), ISR_EncoderA, CHANGE);
  attachInterrupt(digitalPinToInterrupt(pinB), ISR_EncoderB, CHANGE);
}

void loop() {
  // Safe multi-byte variable read operation (Enforcing Atomicity)
  noInterrupts();
  long currentCount = encoderCount;
  interrupts();

  Serial.print("Position Ticks: ");
  Serial.println(currentCount);
  delay(100);
}

void ISR_EncoderA() {
  int stateA = digitalRead(pinA);
  int stateB = digitalRead(pinB);

  if (stateA != stateB) {
    encoderCount++; // A transition and states match opposite -> CW
  } else {
    encoderCount--; // A transition and states match equal -> CCW
  }
}

void ISR_EncoderB() {
  int stateA = digitalRead(pinA);
  int stateB = digitalRead(pinB);

  if (stateA == stateB) {
    encoderCount++; // B transition and states match equal -> CW
  } else {
    encoderCount--; // B transition and states match opposite -> CCW
  }
```

```
}
```

Common Mistakes

- **Omitting the `volatile` Keyword:** Neglecting to declare encoder counter variables as `volatile` causes the compiler to assume the value cannot change spontaneously outside `loop()`. This leads to the processor reading stale cached values.
- **Non-Atomic Multi-Byte Reads:** On an 8-bit architecture (like the ATmega328P), reading a 32-bit `long` takes 4 clock cycles. If an encoder interrupt fires mid-read, the upper and lower bytes become mismatched, causing catastrophic data corruption.
- **The Interrupt Bottleneck:** Processing high-resolution encoder wheels (e.g., 1000 PPR) operating at high RPM via software interrupts can trigger tens of thousands of ISR calls per second. This leaves zero processing cycles for the main program loop, freezing the microcontroller.

Frequently Confused Concepts

- **PPR vs. CPR:** *Pulses Per Revolution (PPR)* refers strictly to the structural physical windows on the encoder wheel. *Counts Per Revolution (CPR)* refers to the decoded software counts after exploiting quadrature edge states. For X4 configurations, $CPR = 4 \times PPR$.
- **Altering Voltage vs. Altering Duty Cycle:** A PWM pin does not modify the physical amplitude of the voltage wave (it remains a 5V or 0V pulse). It strictly modifies the *time domain distribution* of the pulse to alter the integrated average voltage.

Interview / Viva Questions

1. **Q: Why does an encoder require a $90°$ phase shift between Channel A and Channel B?**
   A: The phase shift establishes an asymmetrical state sequence. This phase difference dictates which channel leads the other, allowing the controller to deduce the direction of rotation.
2. **Q: What happens if an encoder spins faster than the microcontroller's execution limits?**
   A: The microcontroller enters an interrupt bottleneck, missing pulse edges. This results in inaccurate position tracking and software lag. This issue is resolved by utilizing hardware QEI peripherals.
3. **Q: Why can't you connect a 5V DC Motor directly to an Arduino PWM output pin without an H-bridge or transistor driver?**
   A: Microcontroller pins are logic-level outputs optimized for signal transmission, limited to roughly 20-40mA. DC motors require much higher current to generate torque. Connecting them directly

draws excessive current, which physically destroys the internal logic structures of the microcontroller pin.

Quick Revision Sheet (1-Page Summary)

Feedback Pipeline (The Quadrature Encoder)

- X2 Encoding Rule: If $A \neq B$ when Pin A changes state → Forward/CW. If $A == B$ → Reverse/CCW.
- X4 Encoding Resolution: Capitalizes on all transitions. Doubles precision over X2, quadruples it over X1.
- Protection Code Blocks:
  `noInterrupts(); long safeCopy = sharedVariable; interrupts();`
- Significance of Sign: A net positive rate of counter change indicates Clockwise motion; a negative rate indicates Counter-Clockwise motion.

Actuation Pipeline (Pulse Width Modulation)

- Core Concept: Simulates continuous analog voltage by adjusting the execution time-on ratio (T_{on}/T_{total}).
- Average Voltage Output: $V_{avg} = Duty\ Cycle \times V_{max}$.
- Arduino Target Resolution: 8-bit depth configuration → Values map linearly between `0` (0% Duty Cycle, Stopped) and `255` (100% Duty Cycle, Full Speed).
- Driver Prerequisite: Always use an H-bridge or power driver to safely separate low-power control signals from high-current motor loads.

Final Takeaways

1. Encoder feedback relies on time-shifted phase tracking. Direction is determined by tracking the relative logic states of Channel A and Channel B during a transition event.
2. Interrupts require strict memory and atomicity management. Use `volatile` declarations and clear `noInterrupts()` boundaries whenever retrieving multi-byte values.
3. PWM alters execution time, not peak voltage amplitude. Motors act as low-pass filters that smooth out high-frequency square pulses into a steady average voltage level.